

Yacc (.y) File Revision Sheet (Ultimate Edition)

This is your ultimate guide for the Yacc (Syntax Analysis) portion of your Compiler Design Viva. Review this carefully to understand exactly how the grammar builds your program!

1. The Big Picture: What is Yacc?

Yacc stands for *Yet Another Compiler Compiler* (often implemented by Bison). It performs **Syntax Analysis**.

- Lex gives Yacc a stream of **Tokens** (words).
- Yacc checks if those tokens form valid "sentences" based on the grammar rules you define (Context-Free Grammar).
- If the syntax is correct, it builds a parse tree. If it's wrong, it throws a syntax error.

2. The 3 Sections of a Yacc File

Just like Lex, the Yacc file is divided by `%%`:

1. **Declarations Section (`%{ ... %}` & `%token`)**: Includes C++ headers, defines global variables, and declares all the tokens received from Lex. Also defines precedence (priority) rules.
2. **Grammar Rules Section (Between `%%` and `%%`)**: Contains Context-Free Grammar (CFG) rules (e.g., `program : program unit`) and the C++ code to execute when a rule matches.
3. **User Code Section**: Contains standard C++ functions like `main()` and `yyerror()` .

3. The Declarations Section

`%token`

```
%token IF ELSE FOR WHILE DO BREAK INT ...
```

This line generates unique integer IDs for every keyword/operator. Yacc shares these IDs with Lex. When Lex returns `IF`, Yacc knows exactly what token ID was received.

The "Dangling Else" Problem (Shift/Reduce Conflict)

```
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
```

Viva Favorite! Why do we need this?

In code like `if (A) if (B) x=1; else y=2;`, Yacc gets confused: does the `else` belong to the first or second `if`? This is called a **Shift/Reduce Conflict**.

By defining `ELSE` below `LOWER_THAN_ELSE`, you are telling Yacc that the `ELSE` token has **higher priority (precedence)**. This forces Yacc to always attach the `else` to the closest, most recent `if` statement, solving the conflict.

4. Grammar Rules & Pseudo-Variables (\$\$)

A rule looks like this:

```
program : program unit
{
    outlog << $1->getnameofsymbol() + "\n" + $2->getnameofsymbol()
    $$ = new symbol_info($1->getnameofsymbol() + "\n" + $2->getnameofsymbol())
}
```

Yacc reads tokens from left to right. When it successfully matches `program unit`, it executes the C++ code inside the braces. We use pseudo-variables to access the tokens:

- **\$1** : Refers to the first element on the right side of the colon (`program`).

- **\$2** : Refers to the second element on the right side (`unit`).
- **\$\$** : Refers to the result on the left side of the colon (the new `program` being created).

How it connects to Lex: Remember how in Lex we created a `symbol_info` object and assigned it to `yylval` ? Here in Yacc, `$1` and `$2` are pointers to those exact `symbol_info` objects! That is why you can call `$1->getnameofsymbol()` to retrieve the exact string that Lex saved!

5. Breaking Down the `main()` Function

The `main()` function ties Lex and Yacc together. Let's look at the crucial lines:

```
int main(int c, char *v[])
```

This allows your program to take command-line arguments. `c` is the count of arguments, and `v[]` contains the actual arguments. (e.g., running `./a.out input.c` means `v[1]` is "input.c").

```
yyin = fopen(v[1], "r");
```

`yyin` is a global file pointer built into Lex. By assigning it to `fopen(v[1], "r")`, you are telling Lex to read its character input from the file you passed in the command line, rather than from the keyboard.

```
outlog.open("22201161_log.txt", ios::trunc);
```

What is `ios::trunc` ?

`ios::trunc` stands for **truncate**. When you open the log file, this flag tells C++ to completely wipe (delete) any old data inside `22201161_log.txt` before writing the new logs. If you ran the compiler twice, you don't want the old run's logs mixed with the new run's logs!

```
yyparse();
```

This is the magic command! Calling `yyparse()` starts the Yacc parser. Yacc will automatically start calling Lex's `yylex()` function repeatedly to grab tokens one by one until it reaches the End-Of-File.

6. The `yyerror()` Function

```
void yyerror(const char *s) {  
    cout << "Error: " << s << endl;  
}
```

If Yacc receives a token from Lex that doesn't fit any of your grammar rules, Yacc throws a syntax error. It automatically calls this `yyerror()` function to print the error message.

7. How to Compile Lex and Yacc Together

Guaranteed Viva Question! You must know how to run these files!

1. `flex 22201161.1` (This generates a C code file named `lex.yy.c` from your Lex file).
2. `bison -d 22201161.y` (This generates the Yacc C code `y.tab.c` and the header file `y.tab.h` which contains all the token IDs that Lex needs).
3. `g++ lex.yy.c y.tab.c -o compiler.exe` (This compiles both C files together into one executable program).
4. `./compiler.exe input.txt` (This runs your compiler on the input code).

8. Pro-Tip: Left Recursion

Bonus Points! Notice how your rules look like this:

`program : program unit` (The word 'program' is on the left side of the right-hand expression).

This is called **Left Recursion**. Yacc/Bison uses a "Bottom-Up" (LALR) parsing method. Bottom-

Up parsers absolutely love Left Recursion because it uses significantly less stack memory than Right Recursion (`program : unit program`). Mentioning this will seriously impress your teacher!